

Sets and Maps

Sets

- A **set** is a collection of elements where each element is unique.
- Common set operations:
 - Union:** The union of two (2) sets ($A \cup B$) is the set that contains all the elements in either set.
Example: $\{1, 3, 5, 7\} \cup \{2, 3, 4, 5\} = \{1, 2, 3, 4, 5, 7\}$
 - Intersection:** The intersection of two (2) sets ($A \cap B$) is the set that contains only the elements common to both sets.
Example: $\{1, 3, 5, 7\} \cap \{2, 3, 4, 5\} = \{3, 5\}$
 - Difference:** The difference of sets A and B ($A - B$) is the set that contains the elements that are in A but not in B.
Example: $\{1, 3, 5, 7\} - \{2, 3, 4, 5\} = \{1, 7\}$
 - Subset:** Set A is a subset of set B ($A \subset B$) if every element of set A is also an element of set B.
Example: $\{1, 3, 5, 7\} \subset \{1, 2, 3, 4, 5, 7\} = \text{true}$
- Java contains three (3) general-purpose set implementations. All are included in the **java.util** package.
 - HashSet** – This stores its elements in a hash table without a guaranteed order upon iteration. This is the best-performing implementation.
 - TreeSet** – This stores its elements in a special type of tree where elements are sorted (natural or custom) during iteration.
 - LinkedHashSet** – This stores its elements in a hash table with a linked list running through it. The order of the elements during the iteration is the same as the order they were inserted into the set.
- Sample codes for sets in Java:
 - To create an empty set:


```
Set a = new HashSet();
Set b = new TreeSet();
Set c = new LinkedHashSet();
```
 - To add items to the set:


```
Collections.addAll(a, "Mark", "Nika", "Mairo", "Kae");
//equivalent to multiple use of add()
```

- To determine the union, intersection, and difference:

```
Set a = new HashSet();
Set b = new HashSet();
Collections.addAll(a, "Mark", "Nika", "Mairo", "Kae");
Collections.addAll(b, "John", "Marco", "Mark");
Set union = new HashSet(a);
Set inter = new HashSet(a);
Set diff = new HashSet(a);
union.addAll(b);
inter.retainAll(b);
diff.removeAll(b);
System.out.println("Union: " + union);
System.out.println("Intersection: " + inter);
System.out.println("Difference: " + diff);
```

Output:

```
Union: [Marco, Nika, Mairo, John, Mark, Kae]
Intersection: [Mark]
Difference: [Nika, Mairo, Kae]
```

- To determine whether a set is a subset of another set:


```
System.out.println(a.containsAll(b));
```
- Curly braces or the **set()** function can be used to implement sets in Python.
- Sample codes for sets in Python:
 - To create an empty set:


```
a = set()
```
 - To initialize a set:


```
a = set(["Mark", "Nika", "Mairo", "Kae"])
b = {"John", "Marco", "Mark"}
#Equivalent to multiple use of add()
```
 - To determine the union, intersection, and difference:


```
a = set(["Mark", "Nika", "Mairo", "Kae"])
b = set(["John", "Marco", "Mark"])
print("Union: " + str(a | b))
print("Intersection: " + str(a & b))
print("Difference: " + str(a - b))
```
 - To determine whether a set is a subset of another set:


```
print(a.issubset(b))
```

Maps

- A **map** is a set of ordered pairs where elements are known as **keys** (identifiers) and **values** (content).
 - A map cannot contain duplicate keys.
 - Each key can map to only one (1) value.

Example (A map of first names):

Key	Value
N	Nika
K	Kae
M1	Mark
M2	Mairo

- Same with sets, there are also three (3) general-purpose map implementations in Java: **HashMap**, **TreeMap**, and **LinkedHashMap**.
- Sample codes for maps in Java:
 - To create an empty map:
`Map<String, String> nameMap = new HashMap<>();`
 - To insert a mapping:
`nameMap.put("M1", "Mark");`
`nameMap.put("M2", "Mairo");`
 - To delete the mapping for a key:
`nameMap.remove("M2");`
 - To replace a key's value:
`nameMap.replace("M2", "Marco");`
 - To retrieve the value based on the key:
`System.out.println(nameMap.get("M1"));`
 - To check whether a map contains a mapping for the specified key or value:
`System.out.println(nameMap.containsKey("M2"));`
`System.out.println(nameMap.containsValue("Mark"));`
 - To retrieve all the keys and values of a map:
`System.out.println(nameMap.keySet());`
`System.out.println(nameMap.values());`
 - To display entries in separate lines:
`for (Map.Entry e : nameMap.entrySet()) {`
 `System.out.println(e.getKey() + ": " +`
 `e.getValue());`
`//Map.Entry is an interface to retrieve entries`

- Maps in Python are known as **dictionaries**. To create a dictionary, use the colon symbol (:) for key-value pairs within the curly braces.
- Sample codes for dictionaries in Python:
 - To create an empty dictionary:
`name_map = { }`
 - To initialize a dictionary:
`name_map = {"M1": "Mark", "M2": "Mairo"}`
 - To insert a mapping:
`name_map["N"] = "Nika"`
 - To delete the mapping for a key:
`del name_map["M2"]`
 - To replace a key's value:
`name_map["M2"] = "Marco"`
 - To retrieve the value based on the key:
`print(name_map["M1"])`
 - To check whether a dictionary contains a mapping for the specified key:
`print("M2" in name_map)`
 - To retrieve all the keys and values of a dictionary:
`print(list(name_map))`
`print(list(name_map.values()))`
 - To display entries in separate lines:
`for x, y in name_map.items():`
 `print(x, y)`

References:

Koffman, E. and Wolfgang, P. (2016). *Data structures: Abstraction and design using Java*. Hoboken: John Wiley & Sons, Inc.

Oracle Docs (n.d.). *Citing sources*. Retrieved from <https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html>

Python Software Foundation (n.d.). *The Python tutorial*. Retrieved from <https://docs.python.org/3/tutorial/index.html>